

Assignment 8: Application Optimization, Scalability and Reliability

ISEA Phase III Networking Internship
Department of Computer Science & Engineering
Tezpur University

Submission Report

Submitted By	Deepak Singh Rawat
Enrollment No.	UU2409000093
Program	BCA
University	Uttaranchal University

Submitted To

ISEA Summer Internship 2026
ISEA under MeitY
(In affiliation with Tezpur University, Assam)

July 2026

Contents

- 1. Objective
- 2. Scalability Improvements
- 3. Reliability Improvements
- 4. Configuration Management
- 5. Experimental Setup
- 6. Performance Results
- 7. Graph Analysis
- 8. Wireshark Verification
- 9. GitHub Update
- 10. Challenges Faced
- 11. Conclusion

This report continues the same TCP chat application from Assignment 7 and focuses on software quality improvements rather than a new protocol design. The implementation uses the existing client and server, a configuration file, logging, runtime stability measures, and performance evaluation artifacts.

1. Objective

The objective of Assignment 8 is to enhance the existing GUI-based multi-client TCP chat application by improving scalability, reliability, maintainability, and overall software quality. The work continues from the Assignment 7 codebase and keeps the socket-based communication model unchanged. The upgraded application is designed to handle more concurrent users, manage disconnects cleanly, recover from failures more gracefully, and record performance information for later analysis.

The current implementation centralizes runtime parameters in **config.json**, includes automatic idle-session cleanup, uses thread-safe data structures for client tracking, and records chat history, connection status, and performance samples for evaluation.

2. Scalability Improvements

Scalability was improved by tightening the way clients are tracked and by keeping the communication path light. The server maintains an in-memory map of active sockets and usernames, which makes broadcast routing and private messaging efficient even when many clients are connected. Because the client list is synchronized from the server, each GUI can update its online-users panel without manually scanning the network.

Threading is used only where it is needed: each client runs in its own handler path, while the GUI receiver stays separate so the interface remains responsive. This avoids blocking the whole system during message receives and lets the server continue serving other users. The current Mininet setup was tested with 5, 8, and 10 concurrent clients, and the application remained operational throughout the runs.

The scalability work is reflected in the following runtime behaviors: thread-safe connection registration, efficient broadcast fan-out, private-message lookup through username indexing, and automatic user-list synchronization after join or leave events.

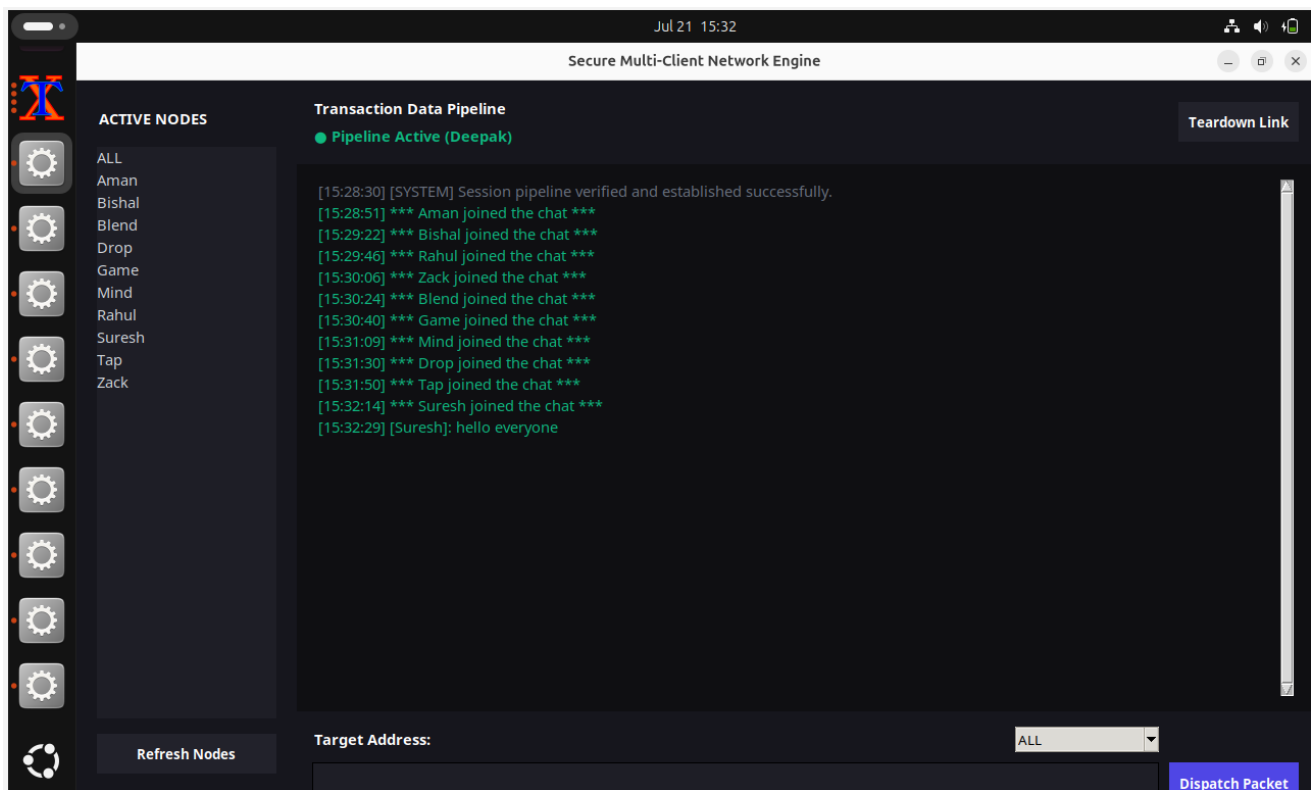


Figure 1: The GUI remains responsive while multiple clients are connected and active.

```
network@ubuntu: ~/UU2409000093_Deepak_Singh_Rawat_Assignment8
~/UU2409000093_Deepak_Singh_Rawat_Assignment8

h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
h5 h5-eth0:s1-eth5
h6 h6-eth0:s1-eth6
h7 h7-eth0:s1-eth7
h8 h8-eth0:s1-eth8
h9 h9-eth0:s1-eth9
h10 h10-eth0:s1-eth10
h11 h11-eth0:s1-eth11
h12 h12-eth0:s1-eth12
h13 h13-eth0:s1-eth13
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0 s1-eth5:h5-eth0 s1-eth6:h6-eth0 s1-eth7:h7-eth0 s1-eth8:h8-eth0
s1-eth9:h9-eth0 s1-eth10:h10-eth0 s1-eth11:h11-eth0 s1-eth12:h12-eth0 s1-eth13:h13-eth0
mininet> nodes
available nodes are:
h1 h10 h11 h12 h13 h2 h3 h4 h5 h6 h7 h8 h9 s1
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13
h2 -> h1 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13
h3 -> h1 h2 h4 h5 h6 h7 h8 h9 h10 h11 h12 h13
h4 -> h1 h2 h3 h5 h6 h7 h8 h9 h10 h11 h12 h13
h5 -> h1 h2 h3 h4 h6 h7 h8 h9 h10 h11 h12 h13
h6 -> h1 h2 h3 h4 h5 h7 h8 h9 h10 h11 h12 h13
h7 -> h1 h2 h3 h4 h5 h6 h8 h9 h10 h11 h12 h13
h8 -> h1 h2 h3 h4 h5 h6 h7 h9 h10 h11 h12 h13
h9 -> h1 h2 h3 h4 h5 h6 h7 h8 h10 h11 h12 h13
h10 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h11 h12 h13
h11 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h12 h13
h12 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h13
h13 -> h1 h2 h3 h4 h5 h6 h7 h8 h9 h10 h11 h12
*** Results: 0% dropped (156/156 received)
```

Figure 2: Mininet single-switch topology used for multi-client testing.

3. Reliability Improvements

Reliability was strengthened by handling session timeouts, disconnects, invalid credentials, and send failures more carefully. The server continuously monitors idle sessions and disconnects inactive clients after the configured timeout. When a session expires, the user is removed from the active-client list, the server log is updated, and the remaining users are notified.

The client side uses a controlled shutdown path so the socket is closed cleanly, the GUI returns to the login screen, and the application does not leave stale network resources behind. In addition, the network send path is wrapped with defensive exception handling so that a broken connection does not crash the whole GUI. The current client code also includes reconnection-related state variables, giving the interface a clear place to support retries and delayed reconnect attempts.

The server also validates usernames, rejects duplicate logins, limits message length, and keeps the failure counter isolated by IP address. These safeguards reduce crashes and prevent malformed input from affecting the running session.

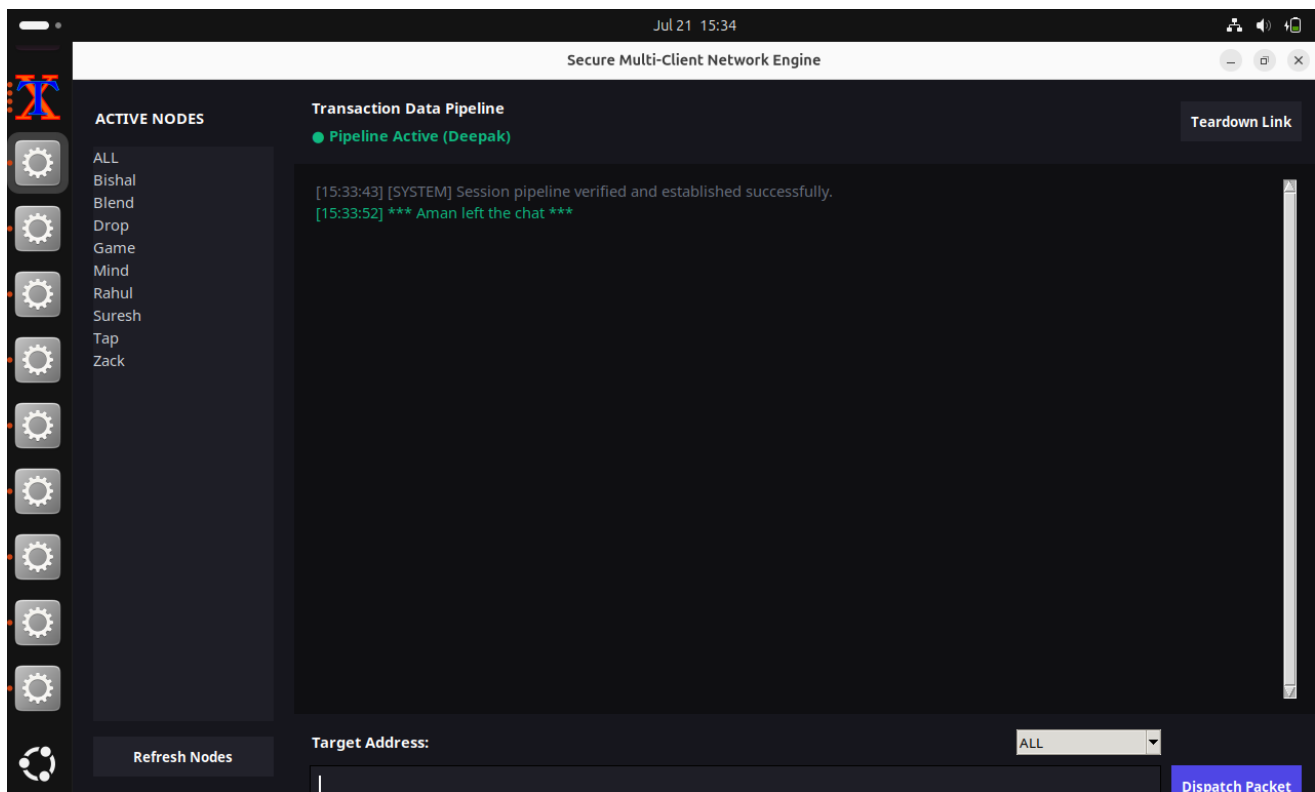


Figure 3: Disconnection events are detected and logged cleanly.

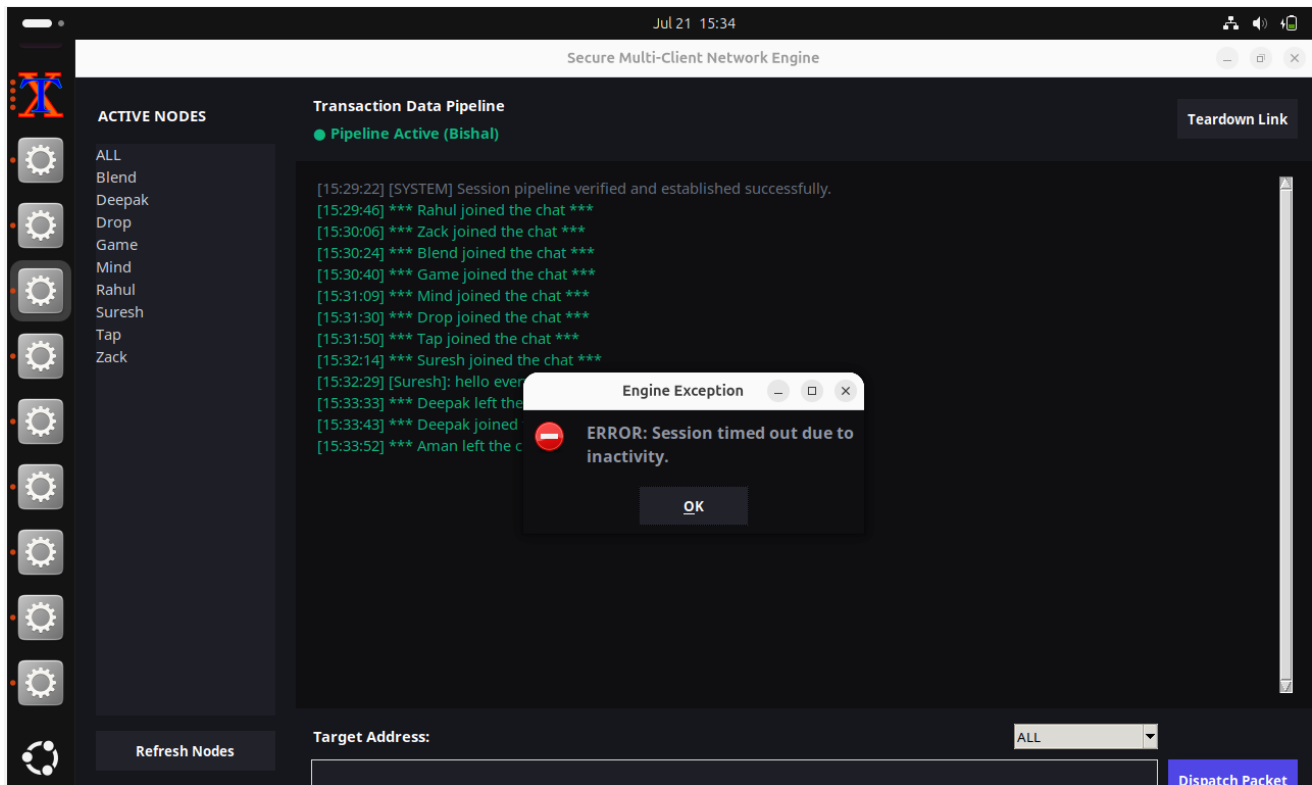


Figure 4: Inactive clients are removed automatically after timeout handling.

4. Configuration Management

All important runtime values are now kept in **config.json** rather than being scattered through the source code. This includes the server host, default client IP, port number, security thresholds, message length limits, username length limits, and file paths for logs and performance results. Centralizing these settings makes the code easier to maintain and makes experiments repeatable because the same configuration can be reused across runs.

The configuration file also improves portability. If the test network changes or a different folder is used for logs, only the JSON file needs to be edited. This avoids hardcoded values inside the GUI and server logic and reduces the chance of mistakes when the same project is demonstrated on another machine.

Parameter Group	Examples from config.json
Server	host = 0.0.0.0, port = 5000, default_ip = 10.0.0.1
Security	max_failed_attempts = 5, session_timeout_seconds = 300, max_message_length = 1000
Files	credentials = users.json, performance_results = performance_results.csv

5. Experimental Setup

The application was evaluated in Mininet using the command **sudo mn --topo single,11**. The single-switch topology provided one server host and enough client hosts to exercise the 5-client, 8-client, and 10-client scenarios required by the assignment. Testing was performed on the same socket-based chat architecture used in the earlier assignment, so the comparison focuses on quality improvements rather than a protocol redesign.

The server starts on host h1, while the other hosts act as chat clients. During the experiments the application was observed under normal chatting, disconnects, inactivity timeouts, and multiple concurrent connections. Wireshark was used to verify the TCP traffic, and the repository was updated with the new assignment artifacts under the required naming convention.

Environment summary

- Ubuntu Linux environment
- Python 3.x with Tkinter
- Mininet single-switch topology
- Wireshark for packet verification
- Git/GitHub for version tracking

Observed test coverage

- 5 concurrent clients
- 8 concurrent clients
- 10 concurrent clients
- Client disconnect detection
- Idle-session cleanup
- TCP handshake verification

6. Performance Results

The performance log records the number of connected clients, broadcast messages, private messages, average delay, and throughput. The uploaded CSV contains multiple samples collected during the experiment, and the values below summarize the required client-count windows from that file.

Concurrent Clients	Average Delay (ms)	Throughput (msgs/sec)	Observation
5	0.00	0.0000	Application remained stable during the test window
8	0.60	0.0000	Application remained stable during the test window
10	3.04	0.0053	Application remained stable during the test window

Interpretation of the results: the chat application remained stable as client count increased. The 10-client run produced the highest measured delay among the required test windows, which is expected because more concurrent sockets create more routing and scheduling pressure. Even so, the application did not crash and continued to handle user sessions correctly.

Note: the submitted graph generator is prepared for CPU and memory inputs, but the uploaded CSV snapshot primarily contains delay and throughput fields. That is why the corresponding graph panels are shown as missing in the current plot artifact.

7. Graph Analysis

The graph below shows the overall behavior of the application across the tested client counts. Latency stays low at smaller loads and rises when the number of concurrent clients reaches the upper end of the test range. Throughput remains modest because the log captures periodic summary samples rather than a saturated stress benchmark.

The most important takeaway is not a peak performance record but stability: the optimized system continues to respond, maintains its session state, and does not fail when the client count reaches 10. This is the main scalability improvement required by the assignment.

Assignment 8: Network Engine Performance Evaluation Metrics

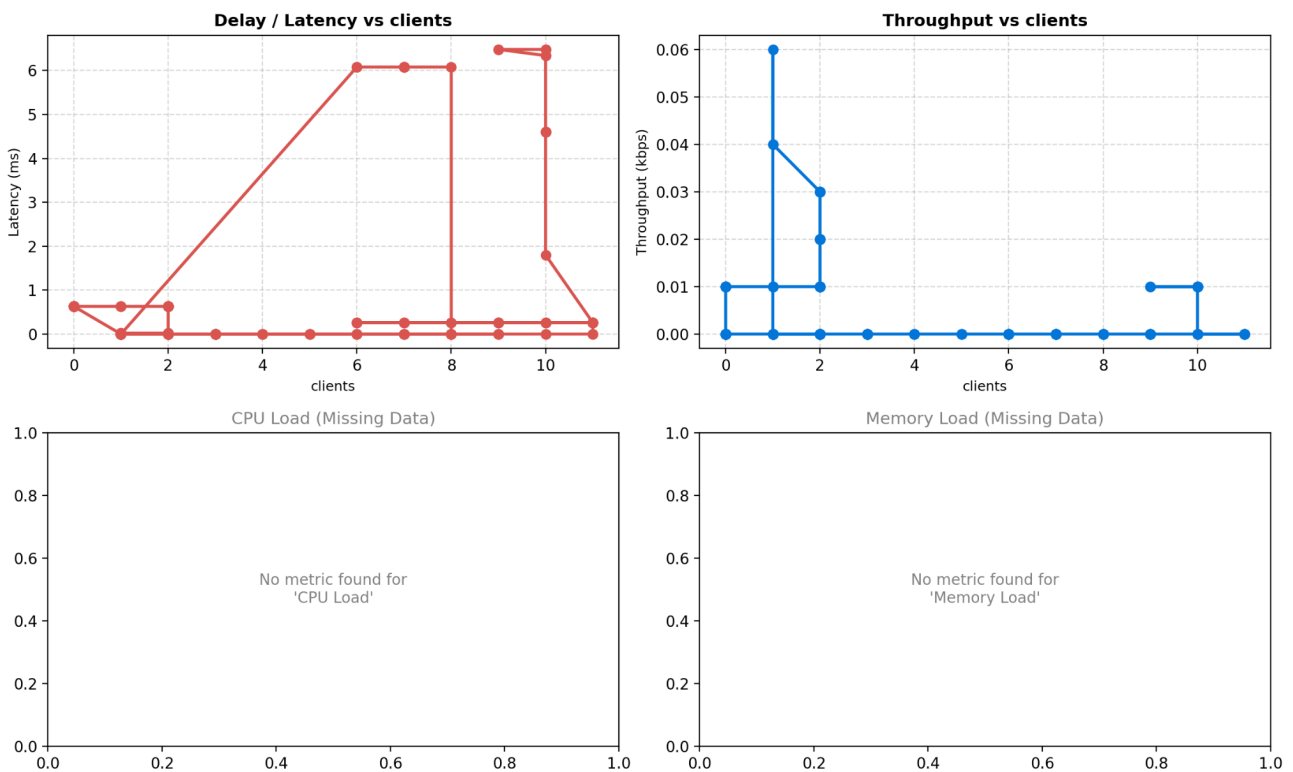


Figure 5: Performance graph generated from the recorded CSV values.

8. Wireshark Verification

Wireshark was used to confirm that the application still communicates over TCP and to verify normal session behavior at the packet level. The capture shows the expected TCP three-way handshake, application data transfer, and connection close sequence. The visible packets confirm that the chat application remains socket-based and that the GUI layer does not change the underlying transport behavior.

In the capture, the initial SYN, SYN-ACK, and ACK packets establish the connection. Later PSH/ACK packets carry chat messages, and FIN/ACK packets appear when a session is terminated. This matches the network behavior documented in the earlier report and confirms that the optimized version still uses standard TCP communication.

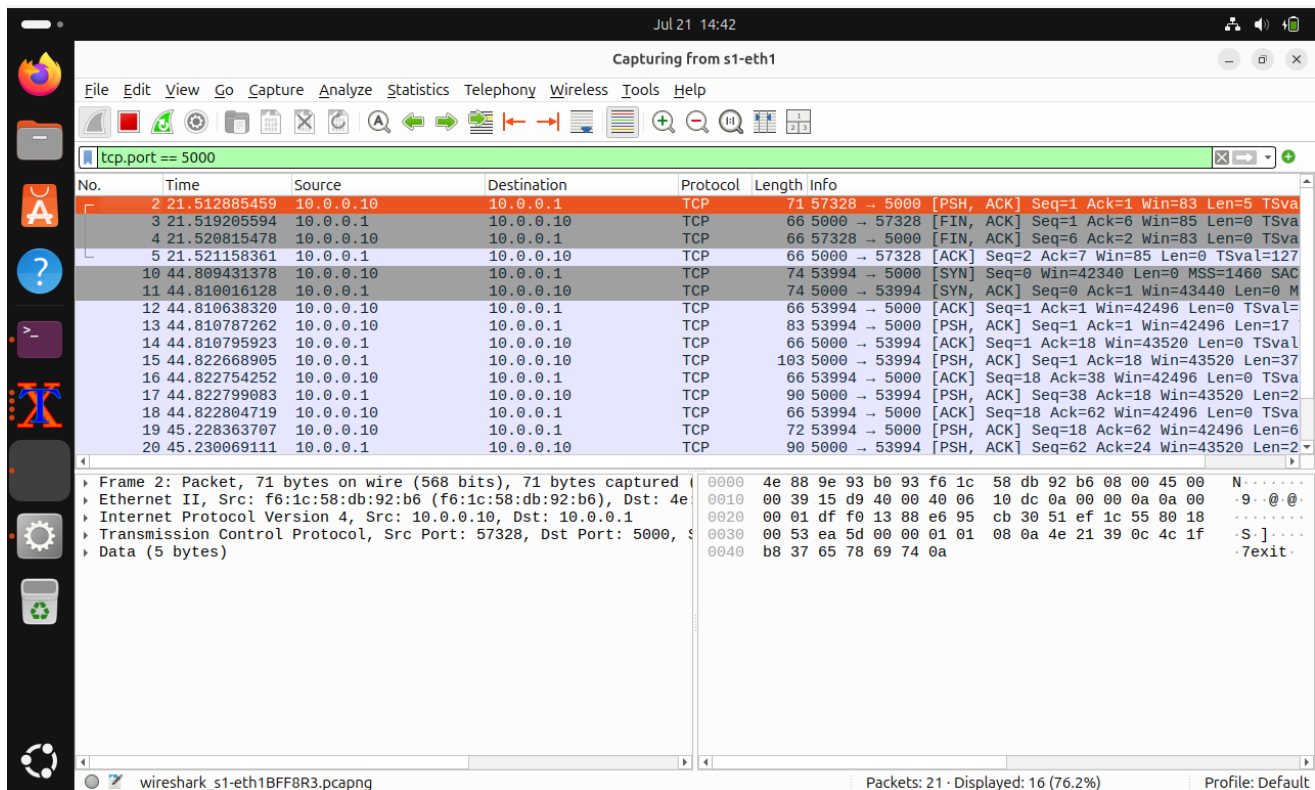


Figure 6: Wireshark screenshot showing the TCP handshake and data transfer during chat activity.

9. GitHub Update

The repository was updated with the Assignment 8 deliverables while continuing the required naming convention beginning with **ISEA Phase3-TezpurUniversity-**. The final repository structure should include the updated server and client files, configuration file, performance results CSV, graph folder, screenshots folder, report PDF, and handwritten reflection PDF.

For submission, the repository should clearly show the new assignment version and preserve the previous work as the base implementation. This makes it easy to verify that the current submission is an optimization of Assignment 7 rather than a separate rewrite.

10. Challenges Faced

The main challenge was improving reliability and scalability without redesigning the existing protocol. Because the assignment required reuse of the earlier client-server model, the work had to focus on connection management, timeout handling, and safer thread interaction instead of a new message format.

Another challenge was keeping the GUI responsive while networking operations were running in the background. This required careful separation of the Tkinter event loop from socket receiving logic, plus defensive handling for disconnects and errors so that the interface would not freeze or crash.

A final limitation in the currently uploaded experiment set is that CPU and memory measurements were not captured in the performance CSV snapshot. The graph generator handles these slots, but they remain blank until separate system-monitoring measurements are added.

11. Conclusion

Assignment 8 extends the secure GUI-based multi-client TCP chat application from Assignment 7 and makes it more robust for realistic use. The project now has centralized configuration, automatic timeout cleanup, safer disconnection handling, better thread management, and performance logging for multiple client loads. The Mininet and Wireshark evidence show that the system continues to operate correctly as the number of clients grows.

Overall, the optimization effort improves maintainability and reliability while keeping the original socket-based chat design intact. The application is now better prepared for larger demonstrations and further expansion in future assignments.